

Synchronizing Key Aspects: Enhancing Vulnerability PoC Reports from Multiple Sources

Wenjing Dang, Lyuye Zhang, Lingxiao Wang, Kaixuan Li, Mengyao Zhao,
Xiaohong Li, and Sen Chen, *Member, IEEE*

Abstract—Proof-of-Concept (PoC) is widely regarded as a practical means to verify vulnerabilities. Moreover, PoCs also play a critical role across multiple stages of vulnerability management. High-quality PoC data is essential for determining whether a patch fully mitigates a vulnerability, confirming whether a vulnerability has migrated across software versions or distributions, and serving as input for automated vulnerability detection and fuzzing systems, etc. However, due to diverse report templates, PoCs from different platforms often suffer from significant information deficiencies. These issues hinder reproducibility and limit their usefulness. Our research demonstrates that integrating data from related PoCs can effectively enhance completeness, improving vulnerability reproduction and facilitating broader dissemination.

In this work, we present the first large-scale study and framework on the complementarity of PoC reports across multiple sources. We began by collecting 456,194 PoC reports from 12 platforms and defining 8 key aspects essential to PoCs. By extracting these aspects with one of the state-of-the-art LLMs, GPT-5, we found that almost all publicly available PoC reports lack at least one key aspect, with approximately 0.034% containing complete information. To address this, we developed a multi-source information fusion framework that supplements missing aspects from CVE entries and related PoC reports, and that judges the complementarity of linked reports before fusion so that behaviorally inconsistent reports are not merged. Finally, we successfully completed 86,595 PoC reports (18.98% of all reports) regarding 8 key aspects. The effectiveness has been further demonstrated by the evaluation of the correctness and reproducibility of completed PoC reports. A comparison against a direct single-LLM completion baseline confirms that fusion is necessary for the exploit-critical aspects, and the completed reports also improve downstream tasks such as retrieval. Moreover, we submitted 20 completed PoC reports to official platforms, including both vulnerability databases and PoC databases. 7 of them have been confirmed, and the others are under review.

Index Terms—Proof-of-Concept, Software Security

I. INTRODUCTION

IN the field of software security, vulnerabilities remain one of the most critical threats to modern information systems. The rapid growth of disclosed vulnerabilities poses formidable challenges for both academia and industry in accurate vulnerability assessment and patch management. Within this landscape, Proof-of-Concept (PoC) reports play a pivotal role. Beyond merely demonstrating the existence of a vulnerability, a well-documented PoC provides concrete evidence about how

the vulnerability can be triggered, under what conditions it manifests, and what observable outcomes indicate successful exploitation. Prior studies have shown that the availability of PoCs is associated with faster remediation [1], [2], and PoCs are also useful for verifying whether patches fully address vulnerabilities that may affect multiple software versions or configurations. In vulnerability detection, PoCs can reduce false positives by supplying concrete trigger conditions and execution evidence [3], [4]. Importantly, PoCs support a broad range of downstream security tasks. They have been used to assess exploitability, estimate vulnerability severity and real-world impact, and help prioritize remediation efforts at scale [2], [5], [6]. They also enable PoC migration and adaptation across software variants, shared code bases, and third-party library dependencies, thereby helping delineate the scope of affected products [7]–[9]. Therefore, PoCs are not merely auxiliary artifacts for vulnerability disclosure; they are essential resources for vulnerability management and cyber threat intelligence, and their value persists even after a vulnerability is patched, when a complete PoC is still needed to confirm whether a fix is correctly backported, delimit the affected version range, and judge exploitability in derived products. As a result, the completeness and reliability of PoC reports directly affect not only PoC reproducibility, but also the effectiveness of downstream tasks such as patch validation, exploitability analysis, and PoC migration, so a report missing key aspects such as the software version, trigger steps, or verification oracle cannot reliably drive any of them [9], [10].

Despite this importance, the quality of public PoC reports varies significantly [11]: the information provided differs greatly across platforms, leaving many reports with substantial gaps that undermine their usability. Yet the problem of missing information in PoCs has not been systematically studied or addressed. To bridge the gap, we investigate the prevalence of missing information in PoC reports from various sources and propose a framework to enrich PoCs by integrating and synchronizing multi-source information. Specifically, we collected 456,194 PoC reports from multiple public repositories and conducted a series of analytical studies. **① Evaluating Information Completeness of PoC Reports.** We identified 8 key aspects that PoC reports should include, derived from the submission templates of multiple platforms. For each aspect, we design an extraction approach based on an LLM to accurately identify and extract key aspects from PoC reports. We found that, only 67.78% of the aspects were present in PoC reports, indicating a significant absence of crucial information. **② Completion of PoC Aspects Based on Multiple Sources.** To mitigate the issue of incomplete information, we developed

Wenjing Dang and Lyuye Zhang contributed equally to this work.

W. Dang, L. Wang, M. Zhao, and X. Li are with Tianjin University, China. K. Li is with the Nanyang Technological University, Singapore. L. Zhang and S. Chen (Corresponding author) are with the Nankai University, China (email: dangwenjing@tju.edu.cn; zh0004ye@e.ntu.edu.sg; wlx_0613@tju.edu.cn; kaixuan.li@ntu.edu.sg; mengyaozhao@tju.edu.cn; xiaohongli@tju.edu.cn; senchen@nankai.edu.cn)

TABLE I
INFORMATION CATEGORIES IN VARIOUS PoC SUBMISSION TEMPLATES.

Sources	# PoCs	Title	Descri.	Dork	Date	Author	Impact	Attack Method	Vendor Link	Patch	Software Link	Product	Tested On	CVE	CWE	CVSS	Ref.
Bugzilla [13]	176,165	✓	○		✓	✓						✓	○	○		○	✓
PHP:Bugs [14]	80,477	✓	✓				✓					✓			✓	✓	✓
ExploitDB [15]	45,471	○		○	✓	✓			✓		○	✓	✓	○			✓
Seebug [16]	41,565	✓	○									✓	✓				✓
CXSecurity [17]	40,377	✓		○		○		✓						○	○		○
Chromium [18]	10,217	✓															✓
Hunter [19]	3,812	✓	✓			○	✓					✓					○
Metasploit [20]	2,101	✓	✓		✓	✓	✓					✓	✓				✓
VulnerLab [21]	1,516		✓			✓			○	○	○	✓			✓		✓
Packet St. [22]	45,757								No Template Required								
Openwall [23]	6,970								No Template Required								
Talos [24]	1,766								No Template Required								

1) ○ means Optional but recommended, ✓ means Required.

2) *Descri.* refers to Description. *Ref.* refers to Reference. *Bugzilla* refers to Red Hat Bugzilla. *VulnerLab* refers to Vulnerability Lab.

a PoC completion tool by leveraging multi-source data. First, we designed a multi-source PoC correlation method to determine whether PoCs from different sources correspond to the same vulnerability. Second, we categorized PoCs into code-based and text-based categories and designed a semantically-based method to supplement PoC reports. Crucially, before merging we judge whether linked reports are behaviorally complementary, so that reports describing different exploit paths of the same vulnerability are not fused. In total, 86,595 PoCs were successfully enriched, and experimental evaluation confirmed the effectiveness of the proposed supplementation strategies. ③ **Correctness and Reproducibility of complemented PoCs.** To assess the correctness of our approach and its effectiveness in facilitating PoC reproduction, a random subset of complemented PoCs was manually evaluated and reproduced. The substantially improved success rate in reproduction, along with enhanced indexing quality, demonstrates the practicality and reliability of the approach. We further compare against a direct single-LLM completion baseline, which confirms that multi-source fusion is necessary for the exploit-critical aspects. ④ **Community Feedback.** Finally, we validated the utility of the completed PoC reports by submitting them to multiple widely recognized databases, including CVE, ExploitDB, CXSecurity, and Seebug. So far, 7 out of 20 have been accepted, while the rest are under review. Beyond these submissions, we further apply the completed PoCs to downstream tasks such as retrieval, affected-version confirmation, and patch-completeness checking; through the latter we found and reported two incomplete fixes that the maintainers accepted and labeled “incomplete fix”. We continue updating our website [12].

In summary, our main contributions are as follows:

- We identify and formalize the problem of PoC key aspects incompleteness, revealing that only about 0.034% of PoC reports contain complete information.
- We propose a unified, generalizable framework for multi-source PoC information fusion and completion, independent of platform templates, whose key design judges the complementarity of linked reports before fusion and which outperforms a direct single-LLM completion baseline, achieving improvement for 86,595 PoC reports and a 91.3% completion correctness rate.
- Our findings provide new insights into PoC data quality and show that completion improves downstream tasks such

as retrieval and affected-version confirmation, establishing a foundation for reliable vulnerability and exploitability research.

II. BACKGROUND AND MOTIVATION

A. Diverse PoC Submission Templates for Multiple Platforms

The sources of PoC reports are diverse, spanning various platforms such as the platform specifically designed for PoC “ExploitDB” and the security vulnerability community platform “CXSecurity” [17]. These platforms offer opportunities for security researchers, from independent individuals to professional security firms, to display and share PoC reports. However, although these platforms share the common goal of facilitating the identification and resolution of security issues, they differ significantly in PoC submission templates and mandatory information. This results in inconsistent PoCs from various data sources associated with the same vulnerability.

To assess the impact of submission templates on PoC report quality, we conducted a study based on prior research [10]. We analyzed PoC submission templates from major platforms, including Red Hat Bugzilla [13], PHP Bug Tracking System [14], ExploitDB [15], Seebug [16], CXSecurity [17], Chromium Monorail [18], Metasploit [20], and Vulnerability Lab [21]. For clarity, we refer to the PHP Bug Tracking System as PHP::Bugs and Chromium Monorail as Chromium. We reviewed the submission templates of these platforms and summarized their required and optional fields in Table I. Our analysis reveals notable discrepancies in submission requirements. For example, ExploitDB and Seebug require detailed test environment information, whereas several other platforms do not. Additionally, all platforms except CXSecurity request information about the affected software or version. In practice, incomplete templates or weak enforcement of submission guidelines further contribute to missing information. Consequently, PoC reports from a single source often contain substantial information gaps, and reports from different sources tend to provide complementary yet different details, limiting their standalone usability and obscuring the relationship between PoCs.

B. Motivating Example

To mitigate these deficiencies, we observed that PoC reports from different sources may reference the same vulnerability,

which may contribute to the complementarity from one source to another based on the varying details. By integrating such diverse information, a more complete and practical PoC can be synthesized, improving both reproduction and indexing. For example, a PoC report from Packet Storm [25] includes only the title, source, keywords, CVE ID, and execution code. This lack of detail likely stems from Packet Storm’s absence of a standardized submission template, resulting in missing information such as the affected software version, operating environment, and reference links. We found another PoC report [26] from ExploitDB, corresponding to the same CVE ID, CVE-2003-0264. It includes details such as the operating platform, reference links, usage methods, and execution results, providing richer information. They target the same vulnerability, thereby sharing useful information. In this example, the additional information contained in the PoC report from another source can be used to supplement the data in the original PoC. Additionally, we examined the NVD entry for CVE-2003-0264, where the “Known Affected Software Configurations” section clearly lists impacted software versions. This version information can be used to reproduce the PoC on the specified software, indicating potential quality improvement of PoC reports in respective platforms by synchronizing the information.

Based on the above observations, we aim to complete the original PoC report by leveraging the extensive information from other related PoCs and the CVE entries corresponding to PoCs. This approach allows us to effectively and comprehensively supplement the missing or insufficient information in the original PoC report.

III. DATA PREPARATION

A. Data Collection

Identifying Data Sources. To gather the most diverse and reliable open-source PoC data possible, we first sourced links tagged with “Exploit” from the “References to Advisories, Solutions, and Tools” section on the NVD, and traced their source platforms. Based on the reference frequency of these links, we selected the top ten candidate sources, including platforms such as ExploitDB and Packet Storm. We further reviewed PoC data sources used in prior studies [2], [9], [10], [27]–[30], identifying eight additional candidate platforms. After merging these candidates and removing duplicates, we obtained sixteen potential sources. We then filtered them by accessibility and data reliability, excluding platforms that had ceased maintenance (e.g., SecurityFocus [31] and SecurityTracker [32]) and platforms without centralized maintenance or submission verification (e.g., GitHub). This process yielded 12 PoC sources, as listed in Table I. We then collected PoC reports from these sources using source-specific crawling or official data access pipelines, followed by preliminary cleaning to remove invalid entries. In total, we collected 456,194 PoC reports. Additional collection details are provided on our project website.

B. Categorizing PoC into Code and Text

PoC reports appear in multiple forms, including executable code, text-based procedures, and other non-textual media.

Following prior work [2] and our observations, we categorize them into code-based, text-based, and other forms. Code-based PoCs contain program code, while text-based PoCs mainly consist of natural-language descriptions. Other forms, such as images, audio, and video, cannot be reliably processed through text parsing. We exclude a report only when it is purely such media with no accompanying text or code; a report that contains text is retained even if it also includes a media attachment. Since non-code and non-text PoCs are relatively rare, and 11 of the 12 sources (except Bugzilla) primarily provide code or text content, our study focuses on code-based and text-based PoC reports. We distinguish these two types using regular-expression-based language detectors for common PoC languages, including C/C++, HTML, Java, JavaScript, Perl, PHP, Python, Ruby, and Shell [2]. Reports matching these language patterns are categorized as code-based; the remaining reports are treated as text-based.

IV. APPROACH AND EMPIRICAL STUDY

We study the following five research questions, and Figure 1 shows how our framework addresses them.

RQ1: What typical aspects does a PoC report comprise and how deficient are they? It aims to uncover the current state of PoC reports information deficiency at the aspect level.

RQ2: Will CVE entries compensate for the deficiency of PoC reports? The potential of PoC reports information supplement from CVE entries is explored.

RQ3: Will Other PoC reports compensate for the deficiency of PoC reports? The potential of PoC reports information supplement from other PoCs is explored.

RQ4: How effective is multi-source completion for PoC reports? Based on CVEs and multi-source PoC reports from various platforms, the completion of PoCs is measured quantitatively based on correctness and reproducibility.

RQ5: What is the community feedback regarding PoC completion? We have submitted the completed PoC reports to CVE, ExploitDB, CXSecurity, and Seebug for feedback.

A. Approach Overview

Our framework takes PoC reports from multiple sources and outputs reports with completed key aspects through four stages, and we organize the empirical study as five research questions (RQ1–RQ5) that examine these stages and their outcomes. First, *Aspect Extraction* prompts an LLM to detect and extract the eight key aspects (defined in Section IV-B1) from each report; this characterizes how incomplete public PoC reports are (RQ1) and yields the structured representation on which all later stages operate. Second, *Multi-Source Linking* associates reports that describe the same vulnerability, through a shared CVE identifier or, when no CVE ID is available, a classifier over the report content, forming the candidate pool for cross-PoC completion (RQ3). Third, *Complementarity Filtering* retains only linked reports whose exploitation behavior is compatible, measured by code-based (UniXcoder) and text-based (Word2Vec) similarity above conservative thresholds, so that only behaviorally consistent reports are fused (RQ3). Fourth, *Aspect Fusion* fills each missing aspect from the

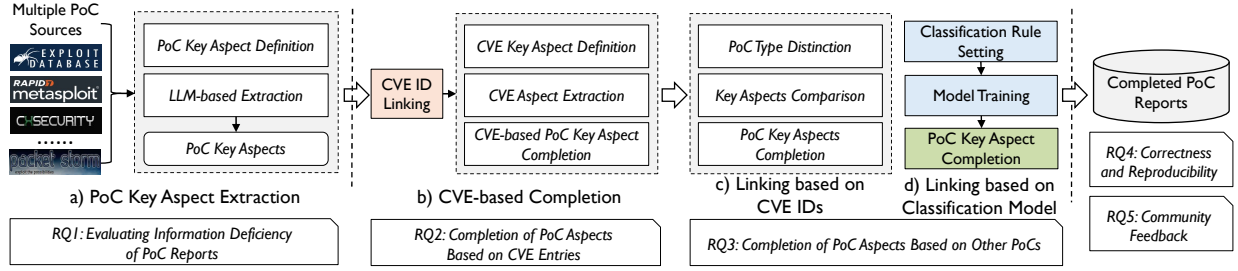


Fig. 1. Overview of our study.

report’s CVE entry when the CVE provides it (RQ2) and otherwise from a complementary report retained by the previous stage (RQ3). The completed reports are then assessed for correctness and reproducibility (RQ4) and validated through community feedback (RQ5).

Algorithm 1 formalizes this procedure, over a PoC corpus P , a CVE database C , and the aspect set A , with $p.a$ the content of aspect a in report p (empty when missing). Here Φ is the LLM extractor, g the classifier that links reports without a shared CVE ID, s the cosine similarity between two reports, and θ its retention threshold (set by report type); the classifier and thresholds are detailed with the stages that use them (Section IV-D2).

The novelty lies in the pipeline as a whole rather than in any single step. Its design is tailored to the characteristics of PoC reports: it extracts key aspects from heterogeneous code and free text, links reports across sources with or without a CVE ID, and, because two PoCs of the same vulnerability may follow different exploit paths, judges behavioral complementarity before fusion. This tailoring fundamentally separates it from direct LLM completion, which fabricates absent content, and from naive same-CVE aspect merging, which ignores exploit-path compatibility. The same pipeline also serves two broader ends: it measures the information deficiency of the current PoC ecosystem and yields high-quality completed PoCs for downstream security applications.

B. RQ1: Evaluating Information Deficiency of PoC Reports

1) **Key Aspects of PoC:** To uniformly characterize the information contained in PoC reports from different sources, we examined the critical elements that PoCs may include, as these directly affect their usability and reference value. Prior work [10] identified important elements for vulnerability reproduction, such as Vulnerable Software Version, Operating System, Config, Trigger method, and Verification. In addition, based on our observations of submission templates across platforms (shown in Section II-A), we summarized the information commonly required in PoC reports. Considering overlaps and missing terms among these elements, we further defined eight critical aspects of PoC reports and grouped them into two categories: Exploit and Basic. Concretely, we retain a field only if it (i) recurs across multiple templates or reproduction taxonomies rather than being platform-specific, (ii) describes intrinsic PoC content rather than an external identifier, and (iii) aids reproduction or indexing. This merges overlapping fields (e.g., vendor, software, and patch links into *Reference*;

Algorithm 1 Multi-Source PoC Aspect Completion

Require: PoC corpus P , CVE database C , aspect set A , code/text thresholds θ_c, θ_t

Ensure: P with completed key aspects

```

1: for each  $p \in P$  do
2:   for each aspect  $a \in A$  do
3:      $p.a \leftarrow \Phi(p, a)$  {Stage 1: LLM aspect extraction}
4:   end for
5: end for
6:  $L \leftarrow \emptyset$ 
7: for all  $p_i, p_j \in P$  do
8:   if  $p_i, p_j$  share a CVE ID and language, or the same software and
      $g(p_i, p_j) = 1$  then
9:     {Stage 2: linking}
10:    if  $p_i, p_j$  are code-based then
11:       $s \leftarrow \cos(\text{UniXcoder}(p_i), \text{UniXcoder}(p_j))$ ;  $\theta \leftarrow \theta_c$ 
12:    else
13:       $s \leftarrow \cos(\text{Word2Vec}(p_i), \text{Word2Vec}(p_j))$ ;  $\theta \leftarrow \theta_t$ 
14:    end if
15:    if  $s \geq \theta$  then
16:       $L \leftarrow L \cup \{(p_i, p_j)\}$  {Stage 3: complementarity filter}
17:    end if
18:  end if
19: end for
20: for each  $p \in P$  and aspect  $a$  with  $p.a = \emptyset$  do
21:   if the CVE of  $p$  provides  $a$  then
22:      $p.a \leftarrow C.a$  {Stage 4: fusion}
23:   else if some  $(p, q) \in L$  has  $q.a \neq \emptyset$  then
24:      $p.a \leftarrow q.a$ 
25:   end if
26: end for
27: return  $P$ 

```

product and version into *Software Version*) and drops platform-specific fields such as Dork and pure identifiers such as CWE and CVSS, of which the CVE ID is reused as a linking key. *Exploit Category:* This category represents the reproduction of the exploit for PoC reports. These 4 aspects are indispensable for the usability of a PoC.

- **Trigger Step** represents the specific steps to activate the vulnerability. Reproducibility is possible with these procedures.
- **Verification Oracle** describes the expected outcome after the vulnerability is triggered, providing a reference for developers using the PoC report and serving as a criterion for whether the PoC can correctly substantiate the vulnerability.
- **Test Platform** indicates the specific system environment or platform on which the PoC operates on.
- **Software Version** specifies the particular affected software with versions. It includes both the software name and detailed version information.

Basic Category: This category serves as the reference value for indexing and searching relevant PoC information.

- **Reference** encompasses links to related information about the PoC report, allowing for a deeper understanding.

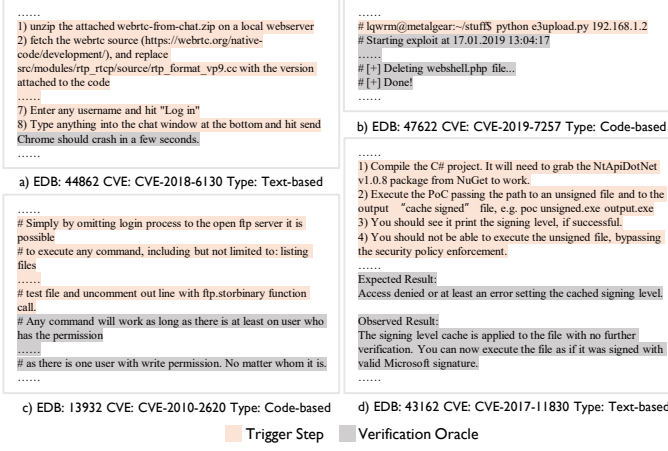


Fig. 2. Examples of Trigger Step and Verification Oracle in PoC reports.

- **Title** refers to the summary of the PoC report. It offers a quick overview of the PoC’s scope and helps users quickly understand the focus and intent of the vulnerability.
- **Author** refers to the publisher name of the PoC report.
- **Publish Time** indicates the date of PoC report publishing.

Overall, these 8 aspects are referred to as the **Key Aspects** of a PoC report. They encompass multi-layered information relevant to vulnerability reproduction and provide essential references associated with PoC reports. The two categories play different roles. The Exploit aspects determine whether a PoC can be reproduced, as they specify how to trigger the vulnerability and how to set up and verify its execution. The Basic aspects instead support indexing and retrieval, so their absence mainly makes a report harder to obtain.

2) **Extraction of Key Aspects**: Based on the eight defined key aspects, we use an LLM to identify and extract all aspects from PoC reports (Stage 1 of Algorithm 1). Specifically, we adopt GPT-5 as the extraction model, providing a unified processing pipeline across different PoC sources and formats. As illustrated by the examples in Figure 2, PoC reports are highly heterogeneous. The same aspect may appear as descriptive text, step-by-step instructions, code comments, or execution outputs. This is especially common for exploit-related aspects such as *Trigger Step* and *Verification Oracle*. Moreover, aspects such as *Software Version* and *Test Platform* often combine natural language with numeric ranges or symbolic notations, making simple pattern matching insufficient. Accordingly, we formulate key aspect extraction as a semantic extraction task. Given a complete PoC report, GPT-5 is prompted to determine whether each predefined aspect is present and to extract the corresponding text when available. By reasoning over the full report context, the model can capture implicit expressions, long-range dependencies, and mixed code–text content commonly found in PoC reports. The complete extraction prompt is shown in Figure 3.

Extraction uses GPT-5 (released August 2025, with a training cut-off of September 2024), with temperature 0 for deterministic output and max_tokens = 4,096. Extracting the eight aspects from all 456,194 PoC reports consumed approximately 1.19×10^9 input and 1.37×10^8 output tokens, corresponding

System. You are a security analyst that extracts structured information from a vulnerability Proof-of-Concept (PoC) report.

Task. Given the full text of one PoC report, decide, for each of the eight key aspects below, whether it is present, and if present, copy the exact corresponding text span verbatim. Judge presence only from what is explicitly stated in the report: do not infer, complete, or fabricate any content. Mark an aspect as present only when it satisfies the inclusion rule and is not excluded by the stated boundary; otherwise mark it absent.

Exploit aspects.

- (1) *Trigger Step*: the concrete, executable steps, commands, requests, or code (with any required input or payload) that activate the vulnerability. A generic description of the vulnerability type or root cause, without actionable steps, does not count.
- (2) *Verification Oracle*: the observable signal confirming successful exploitation, e.g., a crash or segfault, a specific returned error message, unauthorized file or command access, or a privilege change. A coarse impact label such as “buffer overflow” or “remote code execution”, with no observable signal, does not count.
- (3) *Test Platform*: the concrete execution environment in which the PoC is run, i.e., the operating system, distribution or kernel, or runtime, container, or hardware. An affected-product name alone, with no environment information, does not count.
- (4) *Software Version*: the affected software name together with a specific version or version range. A software name with no version, or a version with no software, does not count.

Basic aspects.

- (5) *Reference*: an external link (URL, advisory, commit, or patch) to further information about the PoC or vulnerability. A bare CVE ID with no link does not count.
- (6) *Title*: an explicit title or one-line summary naming the vulnerability or its subject.
- (7) *Author*: the name or handle of the person or group who published or is credited with the PoC.
- (8) *Publish Time*: an explicit publication or disclosure date of the report.

Output. Return only a JSON object mapping each aspect name to {"present": true or false, "text": the extracted span (empty if absent)}.

Fig. 3. The prompt used by GPT-5 for key aspect extraction.

to about US\$2,850 in API cost and roughly 12 hours of wall-clock time with parallel requests (about 3 seconds per report on average). This one-time cost is modest relative to the scale of the corpus and is fully amortized, since the resulting structured dataset is reusable across downstream tasks.

3) **Evaluation of Aspect Extraction**: Regarding evaluation metrics, we use precision ($\frac{TP}{TP+FP}$) and recall ($\frac{TP}{TP+FN}$) to evaluate extraction effectiveness. We randomly sampled 2,500 PoC reports and manually annotated their key aspects using the BIO (Begin, Inside, Outside) tagging scheme [33]. **Labels** denote the number of annotated tokens. To ensure annotation quality, five researchers independently labeled the sampled reports, and the inter-rater agreement measured by Fleiss’ Kappa [34] reached 0.96, indicating highly consistent annotations and a reliable ground truth. The high agreement also reflects the detailed operational definition we set for each aspect. The five researchers followed these definitions as a shared annotation guideline, so disagreements were infrequent. When they did occur, we resolved them by discussion among the annotators until a consensus was reached, and for the few remaining cases (only 11 of the 52,944 labeled tokens) a senior researcher with extensive vulnerability-analysis experience made the final decision. On this benchmark, our GPT-5 extraction reaches 0.96 precision and 0.94 recall overall (Table II), and is strongest on the PoC-specific and irregularly expressed aspects: 0.98/0.98 on both *Trigger Step* and

TABLE II

EFFECTIVENESS OF KEY ASPECTS EXTRACTION. “OURS” IS OUR SEMANTIC-EXTRACTION FORMULATION, REPORTED WITH BOTH GPT-5 AND THE OPEN-WEIGHT QWEN3 TO SEPARATE THE MODEL FROM THE FORMULATION.

Key Aspects	# Labels	BERT-NER		Ours (Qwen3)		Ours (GPT-5)	
		P	R	P	R	P	R
Title	2,472	0.95	0.94	0.96	0.95	0.97	0.96
Publish Time	1,258	0.98	0.97	0.97	0.96	0.98	0.97
Author	878	0.92	0.88	0.96	0.95	0.97	0.97
Test Platform	2,355	0.93	0.93	0.95	0.93	0.96	0.94
Software Version	2,951	0.69	0.67	0.92	0.88	0.94	0.90
Reference	761	0.94	0.99	0.96	0.98	0.97	0.99
Trigger Step	280	0.83	0.84	0.96	0.96	0.98	0.98
Verification Oracle	88	0.75	0.76	0.96	0.96	0.98	0.98
Overall	52,944	0.88	0.87	0.95	0.93	0.96	0.94

1) # Labels indicates the number of annotated tokens for each key aspect in the sampled 2,500 PoC reports; **P** and **R** denote Precision and Recall.

Verification Oracle, and 0.94/0.90 on *Software Version*, where natural language mixes with version patterns. This accuracy is sufficient for the subsequent completion stages.

We further compare against the extraction paradigm used in prior CVE key-aspect studies, which cast the task as named entity recognition with transformer models such as BERT [35]–[38]. Because these models target CVE-oriented entities such as vendor, product, and version and omit PoC-specific aspects such as *Trigger Step* and *Verification Oracle*, a direct comparison would be unfair; we therefore retrain a BERT-NER baseline on our PoC corpus with the same eight aspect labels, keeping its token-level NER formulation while aligning the label space and data distribution. Under this fair setup, BERT-NER reaches only 0.88/0.87 overall, and the gap is largest on the irregular aspects: *Software Version* (0.69/0.67), *Trigger Step* (0.83/0.84), and *Verification Oracle* (0.75/0.76); on regular aspects such as *Reference* and *Publish Time* both methods are comparably strong. This confirms that token-level tagging struggles with the mixed code-text and long-range expressions that full-report semantic extraction handles well.

To separate the effect of the model from that of the formulation, we also run our formulation with the open-weight Qwen3 (*Ours (Qwen3)* in Table II). Holding the semantic-extraction formulation fixed and replacing GPT-5 with Qwen3 changes precision and recall only marginally, from 0.96/0.94 to 0.95/0.93 overall and by at most two points on any aspect. The extraction accuracy therefore stems mainly from formulating the task as full-report semantic extraction, not from the particular LLM.

4) *Information Deficiencies of PoC reports*: After extracting key aspects from PoC reports across 12 sources, we analyzed their distribution, as shown in Table III. Only 157 PoC reports contain all key aspects, accounting for approximately 0.034% of the dataset, indicating that complete PoC reports are extremely rare. Overall, all aspects are missing to varying degrees, with an average presence rate of only 67.78%. Among the *Basic* aspects, *Title*, *Author*, and *Publish Time* have relatively high coverage, each exceeding 85%, whereas *Reference* is lower at 68.36%. Among the *Exploit* aspects, *Trigger Step* appears in only 57.87% of reports, suggesting that many PoCs do not provide detailed usage procedures. *Test Platform* and *Software Version*, both critical for PoC execution, are frequently missing, with presence rates of 72.30% and

TABLE III

DISTRIBUTION OF KEY ASPECTS EXTRACTED FROM PoC REPORTS.

Sources	# PoCs	Exploit Category					Basic Category			
		TrigStep	Verf.	Platform	Version		Title	Author	PubTime	Ref.
Bugzilla	176,165	76.24%	0.06%	100.00%	100.00%		100.00%	100.00%	100.00%	67.97%
PHP::Bugs	80,477	42.26%	0.00%	89.40%	99.99%		99.91%	100.00%	100.00%	26.08%
Packet St.	45,757	53.78%	1.02%	28.99%	55.94%		38.80%	47.78%	26.77%	91.44%
ExploitDB	45,471	31.76%	0.55%	100.00%	58.26%		100.00%	100.00%	99.99%	70.74%
Seebug	41,565	44.01%	0.89%	10.98%	55.54%		22.55%	21.96%	100.00%	96.19%
CXSecurity	40,377	52.99%	0.64%	34.18%	53.94%		100.00%	100.00%	100.00%	92.34%
Chromium	10,217	84.10%	0.01%	19.31%	100.00%		100.00%	100.00%	100.00%	50.21%
Openwall	6,970	55.90%	0.43%	5.06%	26.15%		3.26%	3.92%	3.07%	83.54%
Hunter	3,812	53.33%	0.12%	0.89%	1.63%		100.00%	99.24%	100.00%	96.38%
Metasploit	2,101	0.00%	0.00%	99.67%	0.05%		99.86%	99.81%	99.33%	98.00%
Talos	1,766	53.34%	0.07%	7.81%	99.77%		99.49%	99.49%	99.49%	99.77%
VulnerLab	1,516	98.15%	1.78%	1.25%	63.19%		100.00%	100.00%	100.00%	91.89%
Total	456,194	57.87%	0.41%	72.30%	80.76%		85.31%	86.17%	91.17%	68.36%

80.76%.

Verification Oracle has the lowest presence rate, at only 0.41%. A key reason is that many PoC reports describe exploitation consequences only coarsely (e.g., causing a buffer overflow or memory corruption), rather than providing a concrete and observable post-trigger outcome. Under our definition, a valid oracle must offer an explicit verification signal, such as a crash, a returned error message, access denial, or a privilege change, so that exploit success can be unambiguously determined. Therefore, coarse impact descriptions are not counted as verification oracles. For instance, ExploitDB entries such as nipper-ng 0.11.10 – Remote Buffer Overflow [39], and ICQ 6 ‘Personal Status Manager’ Remote Buffer Overflow [40] provide exploit code and identify the vulnerability category but do not explicitly state a measurable success condition. Since this rate depends on the strict definition, we also report a relaxed definition that counts coarse impact statements (e.g., “buffer overflow”) as weak signals, under which the presence rate rises to about 38%. We keep the strict definition in the main analysis, since only an explicit and observable signal supports unambiguous reproduction, but report both rates so the deficiency is not overstated.

The distribution of aspects also varies substantially across sources. Bugzilla PoC reports show relatively high completeness, with *Test Platform* and *Software Version* reaching 100% coverage, whereas Openwall and Packet Storm have much lower coverage across multiple aspects. This difference is largely explained by platform-specific submission templates: Openwall and Packet Storm lack explicit submission requirements, leading to more severe information deficiencies.

Remarks: All aspects were missing to varying degrees, with key aspects averaging only 67.78% completeness, highlighting widespread information gaps in PoC reports. *Trigger Step* and *Verification Oracle*, both critical for vulnerability reproduction, had particularly low completion rates of 57.87% and 0.41%, greatly hindering reproducibility. This deficiency largely stems from incomplete or missing PoC submission templates.

C. RQ2: Completion of PoC Aspects Based on CVE Entries

As a key vulnerability source, CVE entries offer valuable hints for reproducing and indexing PoC reports. This corresponds to the CVE branch of the fusion stage (Stage 4)

in Algorithm 1. Our completion strategy uses CVE IDs in PoC reports as identifiers to retrieve additional aspects from CVE entries. We first extracted CVE IDs from PoC content, developing source-specific extraction methods due to variations in their placement (e.g., fixed fields in ExploitDB vs. the “References” section in Metasploit’s initialization function). Since not all PoCs include CVE IDs, extraction was limited to those with explicit mentions in designated fields to avoid false positives. After deduplication, we identified 43,108 unique CVE IDs, linked to 76,977 PoC reports.

1) **Aspects from CVE Entries:** Based on prior research [41] and our review of NVD entries, CVE records include key details such as descriptions, affected software, versions, and platforms. Since PoC reports and CVE entries share the same CVE ID, missing information in PoCs can be supplemented from CVE details. This is especially useful, as PoCs aim to validate vulnerabilities, making CVE-provided information directly relevant. We mapped CVE entries to the key aspects defined for PoC reports and found that CVE details on affected software, version ranges, and platforms align closely with the *Software Version* and *Test Platform* in PoCs. Thus, we identified three CVE aspects: **Product**, **Version**, and **Platform**. We use them to verify CVE associations and complete missing PoC aspects. For clarity, CVE aspects are shown in bold and PoC aspects in italics throughout this paper.

2) **Results of Completion by CVE Entries:** We use CVE aspects to enhance PoC completeness. For *Software Version*, we incorporate the **Version** from CVE entries after verifying the **Product**. This unified version data maps directly to the *Software Version* aspect in PoCs. If a PoC lacks version information, we supplement it; if the version list is incomplete, we merge data from multiple sources to ensure all valid vulnerable versions are included. For *Test Platform*, we align it with the **Platform** aspect from CVE entries. Completeness is evaluated by comparing each PoC’s *Test Platform* with CVE data. A PoC is considered incomplete if platform information is missing or not all supported platforms are listed.

As shown in Table IV, our approach enhanced 58,222 PoC reports across 12 datasets, supplementing 168,276 missing aspects. *Software Version* completion far exceeded *Test Platform*, with 58,222 PoCs and 133,493 aspects completed for *Software Version*, compared to 6,482 PoCs and 34,783 aspects for *Test Platform*. This difference stems from the absence of **Platform** information in some CVEs, while **Product** and **Version** are always present. Moreover, vulnerabilities often affect multiple versions, and PoCs rarely list them all, resulting in more completions for *Software Version*. Notably, nearly all PoCs from Metasploit and Hunter had their *Software Version* supplemented, highlighting the initial lack of this detail. Overall, our experiment confirms that leveraging CVE entries is an effective way to improve PoC completeness.

Remarks: We completed 168,276 aspects for 58,222 PoC reports using CVE entries, demonstrating the effectiveness of CVE-based completion.

TABLE IV
COUNT OF COMPLETED PoC REPORTS AND ASPECTS BY CVE ENTRIES.

Sources	CVE	PoC w/ CVE	Test Platform		Software Version	
			PoCs	Aspects	PoCs	Aspects
Bugzilla	190	359	6	32	220	1,410
PHP::Bugs	282	296	9	9	288	846
Packet St.	7,465	8,478	1,203	9,605	7,381	19,455
ExploitDB	24,309	26,517	1,784	8,574	18,790	39,728
Seebug	11,861	14,400	666	2,215	9,292	20,158
CXSecurity	18,334	15,812	1,499	6,387	12,285	26,119
Chromium	1,933	1,960	314	627	1,358	3,108
Openwall	4,089	3,643	348	4,174	3,286	11,171
Hunter	1,666	1,666	8	10	1,665	2,037
Metasploit	1,872	1,982	244	2,986	1,982	6,560
Talos	2,035	1,721	386	99	1,534	2,700
VulnerLab	141	143	15	65	141	201
Overall	43,108	76,977	6,482	34,783	58,222	133,493

1) *Poc w/ CVE* refers to PoC reports with CVE IDs.

D. RQ3: Completion of PoC Aspects Based on Other PoCs

This section covers the linking, filtering, and PoC-based fusion stages (Stages 2 to 4) of Algorithm 1. Mutual completion across PoC reports requires two prerequisites: (i) confirming that PoCs refer to the same vulnerability, and (ii) ensuring that such PoCs describe compatible exploitation behaviors, since one vulnerability may have multiple exploit paths. To address (i), we link PoCs using two types of evidence: identical CVE IDs and semantic correlation between PoC reports (code and free text). To address (ii), we filter linked PoCs by exploitation-level similarity: for code-based PoCs we compute UniXcoder embeddings and cosine similarity, while for text-based PoCs we compute Word2Vec embeddings and cosine similarity, retaining only pairs above the conservative similarity thresholds determined in Step II (Section IV-D2). Applying these two steps allows us to pair PoCs as broadly as possible and complete missing aspects using complementary information from related reports.

1) **Step I: Linking PoCs pointing to the same vulnerability:** In the first step, we establish links between PoC reports using two types of evidence: CVE ID and semantic correlation. **Linking PoCs Based on CVE IDs.** For PoCs linked by a common CVE ID, we leverage the CVE as a marker to associate PoCs from varied sources, as these often pertain to the same vulnerability. However, PoCs describing the same vulnerability may be written in different languages, and their *Exploit* aspects may therefore not be directly comparable or mutually completable. So we first detect the programming language of each PoC and group linked reports by both CVE ID and language. The language detection method is described in Section III-B. As reported in Table V, most multi-source PoCs are text-based (42,895), while among code-based PoCs, C/C++ is the most common (8,113), followed by Ruby, Python, and PHP; Shell and Java are comparatively rare. **Linking PoCs without CVE IDs.** Linking PoCs without CVE IDs is more challenging because it requires understanding both free-text descriptions and source code semantics. To identify PoCs targeting the same vulnerability, we therefore leverage multiple signals, including software name, version, and exploit-logic similarity. Our initial analysis shows that PoCs for the same vulnerability usually affect the same software, although possibly different versions, and often exhibit similar titles and exploit logic.

TABLE V
COUNT OF DIFFERENT TYPES OF PoC REPORTS.

Language Type	CVE	PoC	CVE w/ Link	PoC w/ Link
C/C++	6,243	8,113	1,625	3,676
Ruby	2,324	6,622	1,893	6,207
Python	2,991	5,021	1,603	3,858
PHP	3,448	4,120	1,053	2,153
JavaScript	3,951	3,905	515	951
HTML	2,729	3,826	444	827
Perl	1,800	2,018	161	383
Shell	339	377	32	59
Java	75	80	20	27
Text	29,110	42,895	10,912	26,778
Overall	43,108	76,977	18,258	44,919

Based on this observation, we require linked PoCs to share the same software name, and then use the similarity of their titles and contents as evidence of vulnerability equivalence. To this end, we train a BERT-based classifier [42] to determine whether two PoCs correspond to the same vulnerability. The model takes the titles and contents of two PoCs as input. Training pairs are constructed from PoCs already linked by CVE IDs: pairs with the same CVE ID are treated as positive samples, and pairs with different CVE IDs as negative samples. Since PoCs with CVE IDs account for only 16.9% of the dataset, we use 600 positive and 5,400 negative samples to reflect the real-world distribution, with an 8:1:1 train/validation/test split. The two PoC texts are concatenated with a special separator `[sep]`. The trained model achieves 97.3% accuracy on the test set, supporting the effectiveness of using software names and content similarity for PoC linkage. Because this test set is CVE-derived under a 1:9 ratio, where a majority-class predictor already reaches about 90%, we also evaluated the classifier on a manually labeled set of 200 no-CVE pairs, on which it attains a precision of 0.92, a recall of 0.86, and an F1 of 0.89.

Using this classifier, we identify 41,093 linked PoC pairs among reports without explicit CVE annotations, covering 78,263 PoCs in total. This demonstrates the effectiveness of our method for linking PoCs across sources based on similarity.

Remarks: We successfully linked 28,256 pairs of PoCs by CVE ID and 41,093 pairs of PoCs by model categorization method, indicating a significant result of linking with and without CVE ID.

2) *Step II: Determining Complementarity:* Before performing aspect-level completion across PoC reports, we first assess their behavioral similarity. Although multiple PoCs may point to the same vulnerability, they do not necessarily describe the same exploitation process. In practice, a single vulnerability may admit multiple exploit paths with different triggering conditions, environmental requirements, and verification procedures. For example, CVE-2021-41773 in Apache HTTP Server 2.4.49 has PoCs targeting either file disclosure [43] or remote code execution under additional configurations [44]. Although these PoCs refer to the same CVE, they differ substantially in trigger steps and environmental assumptions. Therefore, directly merging them based only on a shared CVE ID may introduce behaviorally inconsistent

information, making exploitation-level similarity assessment necessary before mutual aspect completion. Specifically, we match PoC reports for similarity by category.

- **Code-Based.** Since the exploit process is difficult to quantify, we approximate it by calculating the semantic similarity between PoC source codes. To do this across various programming languages (C/C++, Ruby, Python, etc.), we use UniXcoder [45], a model designed for code representation learning with a strong semantic understanding of functions, variables, and data structures. Prior work [46] shows UniXcoder outperforms BERT in similar tasks. Specifically, we feed the complete source code of each PoC to UniXcoder and take the encoder output as a fixed-length embedding, and then compute the cosine similarity [47] between the two embeddings. A linked pair is retained for fusion only when this similarity reaches the code threshold selected below.
- **Text-Based.** To determine whether two PoC reports describe similar exploit processes, we utilize the Word2Vec model [48] to represent the textual corpus of PoC reports as vector embeddings. The Word2Vec model effectively captures semantic relationships among words by mapping them into continuous vector spaces, enabling nuanced semantic representation of the textual content. Previous study [49] have demonstrated that Word2Vec achieves superior performance for tasks involving textual similarity measurement. Subsequently, we measure the similarity between these generated vectors using Cosine Similarity, which quantifies the semantic closeness of PoC descriptions.

To validate whether behaviorally similar PoC reports are more likely to provide mutually compatible missing aspects, we manually inspected 100 linked PoC pairs sampled from the RQ3 pipeline and categorized them into four groups: similar and complementary, similar and non-complementary, dissimilar and complementary, and dissimilar and non-complementary. We observed 68 similar-and-complementary pairs, 5 similar-but-non-complementary pairs, 3 dissimilar-yet-complementary pairs, and 24 dissimilar-and-non-complementary pairs. This corresponds to a precision of 93.2% and a recall of 95.8% when using behavioral similarity as an indicator of complementarity. These results show that complementary information is highly concentrated in behaviorally similar PoC pairs, while dissimilar pairs rarely provide reliable mutual completion. Meanwhile, the small number of false positives indicates that similarity is a strong but not sufficient condition, which further justifies our use of conservative thresholds before completion. The details can be found on our website.

Similarity Threshold Determination.

To determine similarity thresholds for PoC completion, we sampled 50 code-based and 50 text-based PoC pairs sharing the same CVE ID (and programming language for code pairs). Three experienced researchers manually labeled similarity, yielding 35 positive and 15 negative code pairs, and 37 positive and 13 negative text pairs. Following prior work [50], we evaluated multiple thresholds by minimizing the false positive rate (FPR) and false negative rate (FNR). As shown in Figure 4a and Figure 4b, thresholds of 0.9 (code)

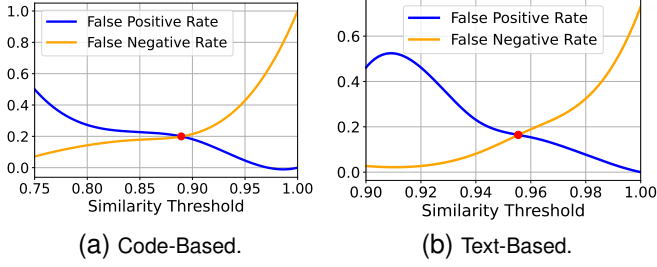


Fig. 4. Similarity threshold selection.

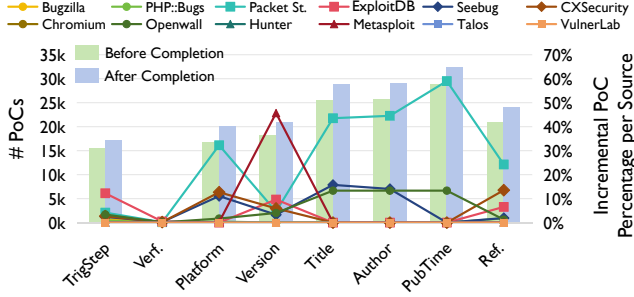


Fig. 5. Overall code-based completion results.

and 0.95 (text) achieve the best trade-off, reducing both FPR and FNR to around 0.1. We therefore adopt 0.9 and 0.95 as the final thresholds for code-based and text-based similarity. A sensitivity check shows that varying either threshold by ± 0.05 changes the completion correctness by at most about three percentage points.

Result Analysis of Code-Based Completion. Following the completion rules of Algorithm 1 and the code threshold above, we performed code-based PoC completion across multiple sources. As shown in Figure 5, our method completed 5,493 PoC reports in total. Due to the scarcity of *Verification Oracle*, only 90 PoCs were completed. For the frequently missing *Trigger Step*, over 1,800 PoCs were supplemented. Additionally, we completed more than 3,000 and 2,500 PoCs for *Test Platform* and *Software Version*, both critical for reproduction.

Regarding the platform deficiency, notably, *Software Version* completion in Metasploit increased by 43%, as this aspect is rarely included. In contrast, platforms like Chromium focus on product-specific vulnerabilities, with limited overlap. Overall, our Code-Based Completion method significantly improved PoC completeness and key aspect coverage across sources.

Result Analysis of Text-Based Completion.

As shown in Figure 6, our approach successfully enhanced a total of 13,354 PoC reports. Specifically, we completed 1,740 to 9,881 PoC reports by supplementing seven missing aspects. The *Test Platform* achieved nearly 10k completions, while the *Author* exceeded 6k. Similar to code-based completion, most completions were concentrated in Packet Storm, ExploitDB, Seebug, and CXSecurity. Since PoC reports from Metasploit are typically in Ruby code format, this source did not contain text-based PoCs. The most notable improvements were observed in Seebug, where *Test Platform*, *Title*, and *Author* completions reached 78.4%, 64.9%, and 71.1%, respectively, due to the significant absence of these aspects.

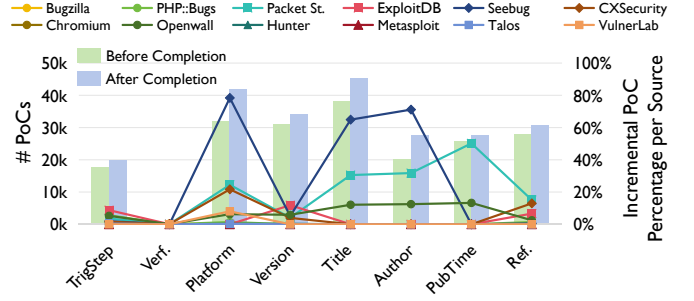


Fig. 6. Overall text-based completion results.

Compared to code-based completion, text-based completion showed more completions and higher completion rates for *Test Platform* and *Software Version*, whereas *Author*, *Publish Time*, and *Reference* were completed at a lower rate. It is likely due to the higher availability of text-based PoCs, while code-based completion was limited by the programming language mappings. In summary, multi-source completion across different PoC types is both feasible and effective, significantly improving the completeness of PoC reports.

Result Analysis of Completion without CVE IDs. In this part, we analyze the complemented aspects on the PoC pairs without CVE IDs that were successfully linked in the previous section, as shown in Figure 7. Our method successfully completed a total of 28,373 PoC reports. The number of PoCs completed varies substantially across the eight aspects, with the highest, 14,936, for *Publish Time*. Notably, although the number of PoC reports containing a *Verification Oracle* was initially low, we were still able to complete the aspect for 1,060 PoC reports. Other important aspects, which significantly aid PoC retrieval, were typically completed for between 5,000 and over 14,000 PoC reports. The completion status varied across different sources. The sources with the most substantial completion were Packet Storm, ExploitDB, Seebug, CXSecurity, and Openwall, with Seebug exhibiting the highest completion rates. This can be attributed to the substantial deficiencies in the original aspects. Completion is also bidirectional across platforms: whether a report gains an aspect depends on what it individually lacks, so every platform both supplies and receives aspects. Even the relatively complete ExploitDB is itself completed by other platforms' PoCs: Figure 7 shows that it gained 512 *Trigger Step* and 2,347 *Software Version* aspects from complementary reports on other platforms. In addition, CVE entries complete it further, adding *Software Version* to 18,790 reports and *Test Platform* to 1,784 (Table IV). The magnitudes are uneven, with more complete platforms acting as net suppliers and sparser ones as net receivers: ExploitDB supplies an aspect in about 65% of its pairs and Seebug in about 30%.

Overall, our method of linking and completing information for PoCs without CVE IDs effectively establishes connections between these previously difficult-to-link PoC reports and enhances their completeness.

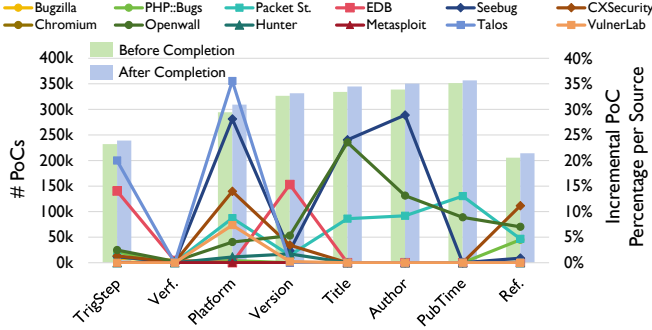


Fig. 7. Overall completion results without CVE IDs.

E. RQ4: Correctness and Reproducibility of Multi-source Completed PoCs

1) **Correctness:** To assess the accuracy of our approach, we randomly selected 10 cases per aspect, totaling 80 cases, and manually verified their correctness. The 80 cases are distributed across nine platforms (ExploitDB 18, Packet Storm 15, Seebug 14, CXSecurity 12, Openwall 8, Metasploit 5, Chromium 4, Bugzilla 2, and VulnerLab 2), spanning both relatively complete sources such as ExploitDB and sparse ones such as Seebug and Packet Storm. We clarify that for each selected PoC, we completed only that specific aspect while keeping all other aspects unchanged, regardless of whether they were already present or still missing. This design ensured that any improvement observed in subsequent reproduction was solely attributable to the completion of that single aspect, rather than to the combination of multiple enhancements. The manual verification involved two steps: ① Checking whether the completed PoC report contained more relevant information than the original. ② Ensuring the completed report did not introduce inconsistencies, like referencing another software product.

To further understand the potential risks of incorrect completion, we conducted a detailed analysis on 80 randomly selected supplemented PoC reports. Among the 80 verified cases, 73 were successfully completed, and 7 failed due to various reasons:

- **Aspect Identification or Extraction Errors (5 cases):** These errors stem from incorrectly identified or incompletely extracted aspects, and are concentrated on the aspects that are hardest to extract: three cases involve *Verification Oracle*, whose success signal is often implicit or described only coarsely, and the remaining two involve *Software Version* and *Reference*. For example, in a *Software Version* case, the PoC [51] listed “AIOCP 1.3.007 and prior versions”, but only “1.3.007” was extracted, causing a completion error. Such inaccuracies do not fundamentally alter the vulnerability semantics and thus have limited impact on practical use, such as vulnerability indexing, retrieval, and dataset aggregation.
- **Similarity Threshold Setting (2 cases):** Both cases occur on *Trigger Step*. Manual inspection of two *Trigger Step* completions [52], [53] showed that, although the paired PoCs shared the same CVE ID and both used C code, their actual exploitation logic differed. The similarity threshold treated them as complementary, so behaviorally incompatible trigger

TABLE VI
REPRODUCTION BASED ON MULTI-SOURCE COMPLETION.

Key Aspects	PoCs	Correct	Successful Reproduction	
			Before Completion	After Completion
Trigger Step	10	8	2	8
Verification Oracle	10	7	2	6
Test Platform	10	10	1	7
Software Version	10	9	0	9
Title	10	10	3	3
Author	10	10	2	2
Publish Time	10	10	2	2
Reference	10	9	2	6
Overall	80	73	14	43

steps were transferred, causing completion errors.

Despite the erroneous cases observed during our sampling check, the overall correctness rate of our approach remains relatively high, indicating strong coverage across completion cases. Further inspection shows that most incorrect completions result in partial or incomplete information rather than entirely false or misleading content. Such issues may reduce the completeness of certain key aspects or occasionally hinder PoC reproduction, but they do not introduce harmful misinformation that could compromise downstream analyses or cause security degradation in real-time systems. Therefore, the potential impact of these errors is limited and manageable.

Remarks: Manual validation shows a 91.3% accuracy across 80 cases, confirming that our method handles most completion scenarios. Its limitations mainly stem from aspect extraction accuracy, which, if improved, will further enhance performance.

2) **Reproducibility:** PoC usability, measured by reproducibility, is key to assessing quality. To evaluate the impact of multi-source completion, we manually reproduced vulnerabilities for the 73 successfully completed PoCs in Section IV-E1. First, we tested reproduction with the original PoCs, then repeated the process with completed aspects. This effort involved eight researchers over 180 man-hours, all with 2 to 6 years of experience (average 3.8) in software security or vulnerability analysis. To ensure fairness, we split them into two experience-balanced groups that reproduced the original and the completed PoCs, respectively, under the same environment, instructions, and criteria, working independently and cross-verifying selected reproductions to reduce individual bias. As shown in Table VI, successful reproductions increased from 14 to 43 after completion. In the Exploit Category, *Software Version* completion yielded the largest gain, raising successful cases from 0 to 9 by identifying the correct affected software, while *Trigger Step* completion increased them from 2 to 8 by clarifying the exploitation steps. *Test Platform* and *Verification Oracle* also contributed, rising from 1 to 7 and from 2 to 6, respectively, by enabling the correct environment setup and confirmation of successful triggering. In the Basic Category, only *Reference* improved reproduction (from 2 to 6), as reference links often point to additional exploitation details, whereas *Title*, *Author*, and *Publish Time* showed no change since they do not directly aid reproduction.

Finally, we analyzed reasons why vulnerabilities in these cases might not be reproducible:

- **Lack of Critical Information (18 cases).** These failures are due to missing or underspecified key aspects, particularly *Trigger Step* and *Software Version*. For example, some reports [54], [55] only provide a high-level vulnerability description without concrete triggering steps or sufficient environment/configuration details. Even with our completion, such vagueness prevents reliably reconstructing the original conditions, and addressing it would require manual inference beyond our automated pipeline.
- **Unavailable Affected Software (9 cases).** A second major cause of failure is the unavailability of vulnerable software versions. Some PoCs target legacy versions that are no longer maintained or distributed, so they cannot be obtained through official or public channels. Thus, even when completion correctly identifies the version, reproduction remains impossible without the corresponding binary or source. For example, PoC [56] specifies “VMware View Portal ≤ 3.1 ”, which is no longer maintained by the vendor and unavailable for download elsewhere. This highlights a practical constraint: accurate metadata alone cannot guarantee reproducibility when the underlying software ecosystem has changed.
- **Unknown Reasons (3 cases).** Even after setting the correct environment and our completion, we found that PoCs, for example [57], could still fail to trigger the vulnerability, highlighting the complexity of real-world vulnerability testing.

In summary, multi-source completion enhances PoC usability and exploit reproduction, and our approach has proven effective. Completing fields like *Title* and *Author* supports indexing on platforms like ExploitDB, enabling searches by title or author. Additionally, completed *Reference* links allow users to access related PoC information directly, greatly improving search and retrieval of relevant PoCs.

3) Comparison with CVE Description Augmentation:

We further compare our approach against a traditional CVE Description Augmentation baseline [35]. Specifically, we first employ an LLM to extract the same set of key aspects from each PoC report. For a PoC associated with a CVE, we then retrieve additional descriptions or reports referring to the same vulnerability and perform aspect-level fusion by filling in missing fields using the corresponding information available in the matched reports. Although this CVE Description Augmentation baseline completes more missing aspects than our method, it yields substantially lower aspect correctness and exploit reproducibility. Concretely, under the same RQ4 setting, its aspect correctness drops to approximately 70.0% (56/80), and the number of successfully reproduced exploits decreases to 27, compared with 91.3% (73/80) correctness and 43 successful reproductions achieved by our approach. Detailed per-aspect results are available on our website. This gap arises because CVE-level augmentation only verifies that two reports reference the same vulnerability and then fuses fields, whereas multiple PoCs of one CVE may follow distinct exploit paths; blindly merging them injects behavior-inconsistent trigger steps and oracles that mislead reproduction despite higher coverage.

TABLE VII
PER-ASPECT COMPLETION RESULTS OF THE DIRECT-LLM BASELINES AND MULTI-SOURCE FUSION UNDER GPT-5. **B1**: LLM DIRECT COMPLETION FROM A SINGLE PoC; **B2**: LLM DIRECT COMPLETION FROM TWO SAME-VULNERABILITY PoCs.

Key Aspect	n	B1			B2			Ours		
		C	F	M	C	F	M	C	F	M
Trigger Step	30	8	20	2	16	14	0	24	3	3
Verification Oracle	30	5	23	2	14	15	1	21	4	5
Software Version	30	6	22	2	21	9	0	27	2	1
Reference	30	4	25	1	25	5	0	27	1	2
Test Platform	30	9	15	6	20	9	1	29	1	0
Title	30	17	11	2	26	4	0	29	1	0
Author	30	3	5	22	28	2	0	28	2	0
Publish Time	30	2	4	24	29	1	0	29	0	1
Overall	240	54	125	61	179	59	2	214	14	12

1) C, F, and M denote Correct, False, and Miss, respectively, out of $n = 30$ completable cases per aspect.

Remarks: During manual exploit reproduction, successful cases increased from 14 to 43 after completion, demonstrating that completing PoC aspects improves reproducibility. Aspects in the Exploit Category proved particularly valuable, while those in the Basic Category had limited impact, with only *Reference* contributing.

4) **Direct-LLM Completion Baseline and Multi-Model Validation:** To test whether a single LLM can complete the missing aspects end-to-end and to isolate each component’s contribution, we compare three configurations: *B1* is LLM direct completion from a single PoC, inferring the missing aspects from the target report alone; *B2* is LLM direct completion from two same-vulnerability PoCs, i.e., our pipeline without the complementarity filter; and *Ours* is the complete complementarity-filtered pipeline. All use the same decoding configuration (temperature 0, max_tokens 4,096). For each aspect, we sample 30 completable cases in which the target PoC lacks the aspect and the linking stage recalls a same-CVE candidate of a priori unknown compatibility, yielding 240 paired cases. Their outputs are anonymously mixed and judged by the RQ4 annotators as *Correct*, *False* (hallucinated or source-mismatched), or *Miss* (abstained), from which we compute $\text{Precision} = \text{Correct} / (\text{Correct} + \text{False})$ and $\text{Recall} = \text{Correct} / n$. This set is distinct from that of Table VI, and a paired McNemar test confirms the significance of the gains of *Ours* over both baselines on the exploit-critical aspects.

As shown in Table VII, the failures of *B1* differ by aspect type. On metadata that cannot be inferred from the report body (*Author* and *Publish Time*), the model tends to abstain, yielding a Miss rate of about 0.77; on aspects that appear inferable (*Trigger Step*, *Verification Oracle*, *Software Version*, and *Reference*), it produces hallucinated and unverifiable content, such as fabricated references, mismatched versions, and invented outputs. The exploit-critical aspects thus fall into the hallucination regime, where the Precision of *B1* is only 0.23. Providing one real candidate (*B2*) removes most fabrication and raises the overall Precision from 0.30 to 0.75; however, without a complementarity judgment, *B2* copies behaviorally incompatible content when two PoCs of the same CVE follow different exploit paths, as in the CVE-2021-41773

TABLE VIII

COMPLETION PERFORMANCE OF THE DIRECT-LLM BASELINES AND MULTI-SOURCE FUSION ACROSS FOUR MODELS. **B1**: LLM DIRECT COMPLETION FROM A SINGLE PoC; **B2**: LLM DIRECT COMPLETION FROM TWO SAME-VULNERABILITY PoCs.

Model	Subset	B1		B2		Ours	
		P	R	P	R	P	R
GPT-5	Overall	0.30	0.23	0.75	0.75	0.94	0.89
	Exploit-critical	0.23	0.22	0.51	0.50	0.87	0.75
Claude	Overall	0.34	0.26	0.77	0.77	0.93	0.88
	Exploit-critical	0.27	0.25	0.54	0.53	0.86	0.74
Gemini	Overall	0.27	0.20	0.72	0.71	0.93	0.87
	Exploit-critical	0.20	0.18	0.47	0.47	0.85	0.73
Qwen3	Overall	0.20	0.15	0.66	0.66	0.92	0.86
	Exploit-critical	0.14	0.13	0.41	0.40	0.84	0.72

1) **P** and **R** denote Precision and Recall. *Exploit-critical* refers to *Trigger Step* and *Verification Oracle*.

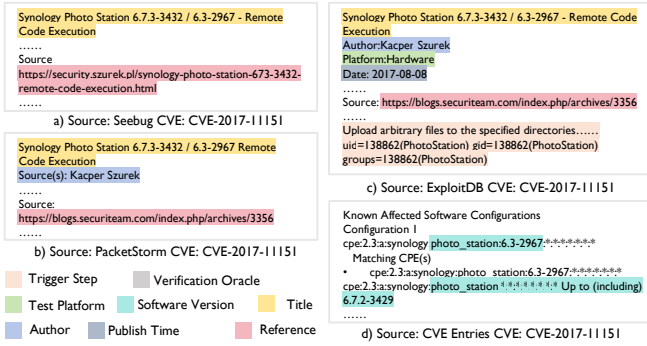


Fig. 8. Example Case of Multiple PoCs and CVE Entries Linked with CVE ID for CVE-2017-11151.

case discussed in Section IV-D2, converting hallucinations into subtler source-mismatch errors. Our complementarity filter rejects such cases and improves the Precision on the exploit-critical aspects from 0.51 to 0.87, at the cost of a slightly lower Recall that reflects a deliberate preference for abstention over unreliable completion. The contribution therefore lies not in whether to fuse, but in determining when not to fuse.

To confirm that this behavior is not model-specific, we repeat the experiment on four models with heterogeneous training corpora: GPT-5 (gpt-5) [58], Claude Opus 4.8 (claude-opus-4-8) [59], Gemini 3 Pro (gemini-3-pro) [60], and the open-weight Qwen3 (Qwen3-235B-A22B-Instruct-2507) [61]. As shown in Table VIII, all four follow the same ordering on the exploit-critical aspects, with Precision increasing from 0.14–0.27 for B1 to 0.41–0.54 for B2 and 0.84–0.87 for Ours. Both proprietary and open-weight models benefit similarly, indicating that the effectiveness stems from the multi-source design rather than from any single model.

5) *Case Study*: To further validate the practical effectiveness of our completion method, we analyze representative real-world cases covering the major completion types. We select two vulnerabilities (linked by CVE ID and by similarity, respectively) and examine six PoC reports in total.

We first study three PoCs from Seebug [62], PacketStorm [63], and ExploitDB [64], all targeting CVE-2017-11151. Since they share the same CVE ID and exhibit clear

TABLE IX

RETRIEVAL EFFECTIVENESS BEFORE AND AFTER COMPLETION.

	Hit@1	Hit@5	Hit@10	MRR
Before completion	0.31	0.55	0.66	0.42
After completion	0.47	0.78	0.88	0.61

implementation relatedness, our tool groups them into one set. As shown in Figure 8, none of the three PoCs initially includes *Software Version*, which prevents straightforward reproduction. By linking the shared CVE ID, the corresponding CVE entry provides the missing *Software Version* information (e.g., 6.7.3-3432 and 6.3-2967), enabling reproduction for all three PoCs. Moreover, the Seebug PoC contains only *Title* and *Reference*, while the ExploitDB PoC provides *Trigger Step* and *Test Platform*, which supplement Seebug and support environment setup and execution. In addition, *Author* information from ExploitDB and Packet Storm completes the missing *Author* in Seebug, and *Reference* links from the three sources are mutually complementary.

We then analyze another group of PoCs from VulnerLab [65], ExploitDB [66], and CXSecurity [67]. Although these PoCs do not contain CVE IDs, our model groups them as the same vulnerability based on identical software version information and high content similarity. Here, the VulnerLab and CXSecurity PoCs lack *Test Platform*, whereas the ExploitDB PoC provides it, allowing us to complete the missing information and improve usability. ExploitDB also contributes more comprehensive references, further enriching the other two PoCs. With *Test Platform* supplemented, the exploit can be reproduced following the PoC report. Overall, these cases demonstrate the effectiveness of our method in completing PoC information by leveraging CVE details and complementary data across multiple sources, thereby enhancing completeness for PoCs with missing aspects.

6) *Downstream Utility of Completed PoCs*: Completion is useful only if it improves the tasks that consume PoC reports. Beyond reproduction, we organize the analysis by three downstream uses, and for each we report both how many reports gain the required inputs and a real vulnerability that the completion makes actionable.

Indexing and retrieval. We evaluate retrievability through a known-item retrieval task. For each of 200 sampled reports, we issue a query composed of its software name, version, and title keywords, and use BM25 over the aspect fields to retrieve the target report, both before and after completion. As shown in Table IX, completion raises Hit@10 from 0.66 to 0.88 and MRR from 0.42 to 0.61, since the supplemented *Title*, *Reference*, *Software Version*, and *Test Platform* provide the searchable content on which retrieval relies. The same aspects also link otherwise isolated reports: completing the aspects of a CVE-less Revive Adserver PoC allows it to be tied to CVE-2021-22873, turning a previously untraceable report into a retrievable one.

Affected-version confirmation. A vulnerability database or scanner often records only one product or a narrow version, while the same flaw affects a wider range of builds that are then left unflagged and exposed. A completed PoC helps confirm the true affected range, since it supplies both the

Trigger Step needed to replay the exploit and the *Software Version* that pins each vulnerable build; completion raises the fraction of reports that provide both inputs from 47% to 78%. Using the completed PoCs, we corrected the affected ranges of several entries and reported five to NVD, of which CVE-2021-4019 has already been accepted [68]: NVD originally listed only Vim (before 8.2.3669), whereas replaying the completed PoC showed that the same heap-overflow also affects Neovim up to 0.7.0, which NVD has since added.

Patch-completeness checking. A released fix is not always complete, and a complete PoC lets a defender re-run the exploit against the patched build to confirm this. This task additionally needs *Verification Oracle*, so the fraction of fully enabled reports stays below 3% because the oracle remains rare even after completion, consistent with the limitation we discuss in the next section; oracle synthesis is left to future work. Where the inputs are present, however, the value is direct: completing the *Trigger Step* of CVE-2021-33624 lets the PoC replay on an unmodified Ubuntu 20.04 kernel and reveals an incomplete backport of the Linux 5.12.13 fix, and the same procedure exposed two incomplete fixes in the AVideo project that we reported and that were accepted and labeled “incomplete fix” [69], [70].

Together, these results show that the completed aspects are exactly what turns an incomplete report into an actionable input for each vulnerability-management task, while the one remaining bottleneck, the *Verification Oracle*, marks the boundary of what completion alone can enable.

F. RQ5: Community Feedback

To evaluate industry recognition of our PoC completion approach, we submitted enhanced PoCs to the CVE database and widely recognized PoC data platforms, including ExploitDB, CXSecurity, and Seebug. We prioritized PoCs with fully completed aspects, except for *Trigger Step* and *Verification Oracle*, ensuring high quality. We manually organized and standardized 20 PoCs according to the submission guidelines and submitted five PoCs to each platform through official channels. For the CVE platform, we hosted the PoCs on our GitHub repository and submitted the links to `CVE.org` for inclusion. Currently, all PoC reports have been received and published by CVE [71]–[75]. For PoC data platforms, as of October 28, 2025, 2 PoCs have been successfully accepted and published on the CXSecurity platform [76], [77], while the rest remain under review. Updates are available on our website [12]. Due to the strict review mechanism of the vulnerability PoC platform, it takes some time from submission to release. We will update the subsequent results on our website. These acceptances show that our newly added aspects and completed PoCs are accepted and welcomed by the community, i.e., the platforms judge them valid contributions.

V. THREATS TO VALIDITY AND LIMITATIONS

A. External Validity

① **PoC format coverage.** We focus on text-based and code-based PoCs, which constitute the vast majority of publicly available PoCs on major platforms [78], [79]. A report is

excluded only when it is purely video, image, or audio with no accompanying text; such purely-media reports (4,894, a small fraction) cannot be parsed for the key aspects, so our method’s effectiveness on them remains to be validated. ② **Mutual completion assumption.** Completion assumes that highly similar PoCs for the same vulnerability are mutually complementary, yet such PoCs may still target different software versions. We mitigate this with conservative similarity thresholds and manual verification, reaching a correctness rate of 91.3%. ③ **Source representativeness.** We study 12 widely used PoC platforms, more than the five or fewer typical in prior work, spanning code storage, PoC databases, and vulnerability platforms, and our framework is not tied to platform-specific templates. We exclude repository-hosted sources such as GitHub (5,843 PoCs) for lacking centralized submission verification and consistent metadata; together with the 4,894 purely-media reports, the excluded parts total 10,737, about 2.3%, so our reported rates characterize the centrally submitted PoC ecosystem of 456,194 reports rather than the entire public one. ④ **Potential data leakage.** Some evaluated PoCs may appear in GPT-5’s training data. Its effect is limited, because GPT-5 serves only as an aspect extractor, a span-copying task whose answer already lies in the report, so a memorized PoC cannot inject fabricated content; and the completion gains come from cross-source complementarity that no model can memorize, since a memorized PoC does not reveal which other PoC it may be safely fused with. Consistently, the same $B1 < B2 \ll \text{Ours}$ ordering holds across four models including the open-weight Qwen3 (Table VIII). ⑤ **Availability for newer vulnerabilities.** For newly disclosed vulnerabilities with few related PoCs, completion still applies: the CVE/NVD record is populated with affected products, versions, and platforms shortly after publication, so CVE-based completion can supply *Software Version* and *Test Platform* even for a single-PoC vulnerability, while further PoCs that later accumulate across platforms can be linked by the no-CVE linker (Section IV-D). Completion is unavailable only for an isolated PoC with neither a CVE record nor a related PoC, a transient early-disclosure condition.

B. Internal Validity

① A major threat is subjectivity in setting similarity thresholds. To mitigate this, we selected the thresholds by minimizing FPR and FNR on manually labeled pairs and verified their robustness with a sensitivity check (Section IV-D2). ② Another threat concerns the accuracy of manual aspect annotations used to train and evaluate the extraction model. To reduce individual bias, five experienced researchers collaboratively performed the annotations to improve reliability. ③ Multi-source supplementation may introduce incorrect information; our audit of 80 completions found five extraction errors and two similarity-pairing errors. To reduce false positives we apply the conservative similarity thresholds selected in Step II (Section IV-D2), a two-step link-then-filter strategy, and GPT-5-based extraction. These measures cannot eliminate all errors at scale but substantially reduce misinformation injection.

C. Limitations

① The first limitation lies in the accuracy of aspect extraction and completion. Even though we employed state-of-the-art models for key aspect extraction, the final completion results still contained about 8.7% errors. Such inaccuracies are difficult to eliminate given the irregular and noisy nature of PoC data. We acknowledge this limitation and will continue exploring more robust extraction and validation strategies (e.g., dynamic verification, cross-source consistency checks) to further reduce error rates in future work. ② A data limitation lies in the very low presence of some key aspects in public PoC reports (e.g., the Verification Oracle). The lack of initial data also limits our completion effect. Our current work focuses on revealing this deficiency and supplementing existing aspects via multi-source data fusion, rather than generating verification outputs. We acknowledge this gap, and as part of our future work, we plan to leverage LLMs in combination with symbolic execution to simulate their execution and infer expected runtime behaviors (e.g., error messages, system call patterns, or output traces) that serve as candidate oracles.

VI. RELATED WORK

A. Studies on PoC Reports

PoC reports complement vulnerability disclosures by providing concrete reproduction methods and trigger examples. High-quality PoC data underpins many downstream applications. For example, PoCs have been used to predict vulnerability exploitability [2], [6], [80]. Suciu et al. [2] observed that PoCs are often malformed programs mixing code with free-form text, which limits the applicability of existing analysis techniques. PoCs are also widely used to validate vulnerabilities and reduce false positives in detection systems [7], [9], [81], to guide fuzzing for more efficient vulnerability discovery [82], [83], and to identify affected Linux kernel versions [84], [85]. These studies show that PoCs are not only valuable for validation but also serve as essential inputs that improve the effectiveness of vulnerability management and defense tasks. Despite their importance, prior work has paid limited attention to PoC report quality itself, and missing or incomplete PoC information has not been systematically studied. Motivated by this gap, we shift the focus from downstream usage to PoC completeness: we quantify missing aspects in PoC reports and evaluate multi-source information fusion to improve their completeness and usability.

B. Studies on Vulnerability Reports

Vulnerability reports are a critical information source for vulnerability management and security research, and a large body of work has studied how to analyze and leverage them. Early studies estimated exploitability directly from disclosure data, e.g., predicting exploitability from vulnerability descriptions [5], [86] or co-exploitability across vulnerabilities [87]. Another line of work extracts structured information from vulnerability texts using NLP. For example, Dong et al. [88] applied named entity recognition and relation extraction to identify affected libraries; Guo et al. [89] combined CoreNLP

with rules to extract descriptions and infer supplementary key aspects; and Sun et al. [41] used BERT with a question-answering framework to capture fine-grained information. These studies demonstrate the effectiveness of NLP in uncovering critical vulnerability attributes. Moreover, SemFuzz [90] used CVE descriptions to guide automatic PoC generation, highlighting the strong linkage between vulnerability reports and PoC data and supporting the feasibility of our approach. Tang et al. [91] further modeled extreme vulnerability disclosure behaviors in industrial settings, showing how disclosure patterns affect attack surfaces and defense.

Beyond single-source analysis, several studies [92]–[96] collected vulnerability reports from multiple sources and revealed substantial cross-source inconsistencies. Such discrepancies can mislead vulnerability prioritization and patching, reduce the reliability of automated assessments, and weaken security operations, highlighting the need for multi-source aggregation and reconciliation. While differing from our setting and objectives, these works motivate our study of improving PoC report completeness through multi-source fusion.

C. Techniques Along Our Pipeline

Our pipeline builds on three lines of technique. Key-aspect extraction has been cast as named entity recognition with transformer models such as BERT [35], [41], [42], while we adopt an LLM extractor for the more heterogeneous PoC text. Similarity has been studied through code clone detection [97], [98] and corpus-based text similarity [99], usually within a single language or modality, whereas we combine UniXcoder [45] for code and Word2Vec [48] for text. Prior multi-source studies mainly expose inconsistencies [92], [95], whereas we complete the missing aspects. To our knowledge, no prior work combines them for PoC completion.

VII. CONCLUSION

In this study, we collected PoC reports from 12 sources and identified 8 key aspects, finding that only 67.78% of these aspects were present on average. To enhance completeness, we supplemented missing aspects using CVE-linked PoCs, successfully enriching 58,222 PoCs with 168,276 additional aspects. For PoCs with CVE IDs, we separate code-based and text-based cases and apply tailored similarity methods; for PoCs without CVE IDs, we use a model-based classifier to link related PoCs, completing 28,373 reports. By judging complementarity before fusion, our framework avoids merging behaviorally inconsistent reports and outperforms a direct single-LLM completion baseline. Our method achieved a 91.3% accuracy rate in manual validation and increased the number of successfully reproducible PoCs from 14 to 43. The completed reports also improve downstream tasks such as retrieval. Additionally, we submitted 20 completed PoCs to multiple platforms, with 7 already accepted and published.

REFERENCES

- [1] D. Votipka, R. Stevens, E. M. Redmiles, J. Hu, and M. L. Mazurek, "Hackers vs. Testers: A Comparison of Software Vulnerability Discovery Processes," in *2018 IEEE Symposium on Security and*

- Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA*. IEEE Computer Society, 2018, pp. 374–391. [Online]. Available: <https://doi.org/10.1109/SP.2018.00003>
- [2] O. Suciu, C. Nelson, Z. Lyu, T. Bao, and T. Dumitraş, “Expected exploitability: Predicting the development of functional vulnerability exploits,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 377–394.
 - [3] J. Krupp and C. Rossow, “{teEther}: Gnawing at ethereum to automatically exploit smart contracts,” in *27th USENIX security symposium (USENIX Security 18)*, 2018, pp. 1317–1333.
 - [4] D. Perez and B. Livshits, “Smart contract vulnerabilities: Vulnerable does not imply exploited,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 1325–1341.
 - [5] M. Bozorgi, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond heuristics: learning to classify vulnerabilities and predict exploits,” in *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2010, pp. 105–114.
 - [6] Y. Wang, C. Zhang, X. Xiang, Z. Zhao, W. Li, X. Gong, B. Liu, K. Chen, and W. Zou, “Revery: From proof-of-concept to exploitable,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1914–1927.
 - [7] J. Dai, Y. Zhang, H. Xu, H. Lyu, Z. Wu, X. Xing, and M. Yang, “Facilitating vulnerability assessment through poc migration,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 3300–3317.
 - [8] Z. Jiang, S. Gan, A. Herrera, F. Toffalini, L. Romero, C. Tang, M. Egele, C. Zhang, and M. Payer, “Evocatio: Conjuring bug capabilities from a single poc,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1599–1613.
 - [9] S. Kwon, S. Woo, G. Seong, and H. Lee, “OCTOPOCS: automatic verification of propagated vulnerable code using reformed proofs of concept,” in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2021, pp. 174–185.
 - [10] D. Mu, A. Cuevas, L. Yang, H. Hu, X. Xing, B. Mao, and G. Wang, “Understanding the reproducibility of crowd-reported security vulnerabilities,” in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 919–936.
 - [11] “Qualys Survey of Top 10 Exploited Vulnerabilities in 2023 — Qualys Security Blog,” <https://blog.qualys.com/qualys-insights/2023/09/26/qualys-survey-of-top-10-exploited-vulnerabilities-in-2023>, 2024, (Accessed on 05/11/2024).
 - [12] “PoC completion,” <https://sites.google.com/view/poccompletion/>, 2024, (Accessed on 06/02/2024).
 - [13] Bugzilla, “Red Hat Bugzilla Main Page,” <https://bugzilla.redhat.com/>, 2024, (Accessed on 05/11/2024).
 - [14] Phpbugs, “PHP :: Bugs homepage,” <https://bugs.php.net/>, 2024, (Accessed on 05/11/2024).
 - [15] OffSec, “Exploit Database,” <https://www.exploit-db.com/>, 2024, (Accessed on 05/11/2024).
 - [16] Seebug, “Seebug,” <https://www.seebug.org/>, 2024, (Accessed on 05/11/2024).
 - [17] Cxsecurity, “CXSECURITY.COM Free Security List,” <https://cxsecurity.com/>, 2024, (Accessed on 05/11/2024).
 - [18] Chrome, “Monorail,” <https://bugs.chromium.org/>, 2024, (Accessed on 05/11/2024).
 - [19] Hunter, “Huntr - The world’s first bug bounty platform for AI/ML,” <https://huntr.com/>, 2024, (Accessed on 05/11/2024).
 - [20] Metasploit, “Metasploit — Penetration Testing Software, Pen Testing Security — Metasploit,” <https://www.metasploit.com/>, 2024, (Accessed on 05/11/2024).
 - [21] Vullab, “VULNERABILITY LAB - SECURITY VULNERABILITY RESEARCH LABORATORY - Best Independent Bug Bounty Programs, Responsible Disclosure & Vulnerability Coordination Platform - INDEX,” <https://www.vulnerability-lab.com/>, 2024, (Accessed on 05/11/2024).
 - [22] Packet, “Packet Storm,” <https://packetstormsecurity.com/>, 2024, (Accessed on 05/11/2024).
 - [23] Openwall, “Openwall - bringing security into open computing environments,” <https://www.openwall.com/>, 2024, (Accessed on 05/11/2024).
 - [24] Talos, “Cisco Talos Intelligence Group - Comprehensive Threat Intelligence,” <https://www.talosintelligence.com/>, 2024, (Accessed on 05/11/2024).
 - [25] Packet Storm, “SLMail 5.1.0.4420 Remote Code Execution,” <https://packetstormsecurity.com/files/161526>, 2024, (Accessed on 05/11/2024).
 - [26] Exploit Database, “Seattle Lab Mail (SLmail) 5.5 - POP3 ‘PASS’ Remote Buffer Overflow (1),” <https://www.exploit-db.com/exploits/638>, 2024, (Accessed on 05/11/2024).
 - [27] Z. Chen, X. Hu, X. Xia, Y. Gao, T. Xu, D. Lo, and X. Yang, “Exploiting Library Vulnerability via Migration Based Automating Test Generation,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.
 - [28] M. Yuan, Y. Lee, C. Zhang, Y. Li, Y. Cai, and B. Zhao, “Raproducer: Efficiently diagnose and reproduce data race bugs for binaries via trace analysis,” in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 593–606.
 - [29] R. Paul, A. K. Turzo, and A. Bosu, “Why security defects go unnoticed during code reviews? a case-control study of the chromium os project,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1373–1385.
 - [30] S. Gupta, B. B. Gupta, and P. Chaudhary, “Hunting for DOM-Based XSS vulnerabilities in mobile cloud-based online social network,” *Future Generation Computer Systems*, vol. 79, pp. 319–336, 2018.
 - [31] SecurityFocus, “SecurityFocus,” <http://www.securityfocus.com/>, 2024, (Accessed on 05/11/2024).
 - [32] SecurityTracker, “SecurityTracker,” <http://www.securitytracker.com/>, 2024, (Accessed on 04/24/2024).
 - [33] L. A. Ramshaw and M. P. Marcus, “Text chunking using transformation-based learning,” in *Natural language processing using very large corpora*. Springer, 1999, pp. 157–176.
 - [34] Wikipedia, “Fleiss’s kappa - wikipedia,” 6 2006, [Online; accessed 2026-01-23]. [Online]. Available: https://en.wikipedia.org/wiki/Fleiss%27s_kappa
 - [35] H. Guo, Z. Xing, S. Chen, X. Li, Y. Bai, and H. Zhang, “Key aspects augmentation of vulnerability description based on multiple security databases,” in *2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2021, pp. 1020–1025.
 - [36] K. Sumoto, K. Kanakogi, H. Washizaki, N. Tsuda, N. Yoshioka, Y. Fukazawa, and H. Kanuka, “Automatic labeling of the elements of a vulnerability report cve with nlp,” in *2022 IEEE 23rd International Conference on Information Reuse and Integration for Data Science (IRI)*. IEEE, 2022, pp. 164–165.
 - [37] F. Hashemi Chaleshtori and I. Ray, “Automation of vulnerability information extraction using transformer-based language models,” in *European Symposium on Research in Computer Security*. Springer, 2022, pp. 645–665.
 - [38] W. Hu and V. L. Thing, “Cpe-identifier: Automated cpe identification and cve summaries annotation with deep learning and nlp,” *arXiv preprint arXiv:2405.13568*, 2024.
 - [39] G. Levin, “nipper-ng 0.11.10 - remote buffer overflow (poc) - linux remote exploit,” 11 2019, [Online; accessed 2026-03-10]. [Online]. Available: <https://www.exploit-db.com/exploits/47673>
 - [40] L. Juranic, “Icq 6 - ‘personal status manager’ remote buffer overflow (poc) - windows dos exploit,” 4 2008, [Online; accessed 2026-03-10]. [Online]. Available: <https://www.exploit-db.com/exploits/31656>
 - [41] J. Sun, Z. Xing, X. Xia, Q. Lu, X. Xu, and L. Zhu, “Aspect-level information discrepancies across heterogeneous vulnerability reports: Severity, types and detection methods,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 2, pp. 1–38, 2023.
 - [42] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
 - [43] lorddemon, “Github - lorddemon/cve-2021-41773-poc,” [Online; accessed 2026-02-27]. [Online]. Available: <https://github.com/lorddemon/CVE-2021-41773-PoC>
 - [44] L. Souza, “Apache http server 2.4.49 - path traversal & remote code execution (rce) - multiple webapps exploit,” 10 2021, [Online; accessed 2026-02-27]. [Online]. Available: <https://www.exploit-db.com/exploits/50383>
 - [45] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, “Unixcoder: Unified cross-modal pre-training for code representation,” *arXiv preprint arXiv:2203.03850*, 2022.
 - [46] F. Ebrahim and M. Joy, “Semantic similarity search for source code plagiarism detection: An exploratory study,” in *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*, 2024, pp. 360–366.
 - [47] J. Ye, “Cosine similarity measures for intuitionistic fuzzy sets and their applications,” *Mathematical and computer modelling*, vol. 53, no. 1-2, pp. 91–97, 2011.
 - [48] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.

- [49] X. Jin, S. Zhang, and J. Liu, "Word semantic similarity calculation based on word2vec," in *2018 International Conference on Control, Automation and Information Sciences (ICCAIS)*. IEEE, 2018, pp. 12–16.
- [50] X. Zhan, L. Fan, S. Chen, F. We, T. Liu, X. Luo, and Y. Liu, "Atvhunter: Reliable version detection of third-party libraries for vulnerability identification in android applications," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1695–1707.
- [51] Exploit Database, "AIOCP 1.3.x - 'cp_codice_fiscale.php' SQL Injection," <https://www.exploit-db.com/exploits/28933>, 2024, (Accessed on 05/11/2024).
- [52] —, "Linux Kernel (Debian 78910 Fedora 232425 CentOS 5.35.116.06.87.2.1511) - 'ldso_hwcap Stack Clash' Local Privilege Escalation," <https://www.exploit-db.com/exploits/42274>, 2024, (Accessed on 05/11/2024).
- [53] Packet Storm, "Linux Kernel offset2lib Stack Clash," <https://packetstormsecurity.com/files/143204>, 2024, (Accessed on 05/11/2024).
- [54] —, "Microsoft Edge Chakra JIT BoundFunction::NewInstance Bug," <https://packetstormsecurity.com/files/148530>, 2024, (Accessed on 05/11/2024).
- [55] Exploit Database, "Microsoft Edge Chakra JIT - BoundFunction::NewInstance Out-of-Bounds Read," <https://www.exploit-db.com/exploits/45012>, 2024, (Accessed on 05/11/2024).
- [56] Packet Storm, "VMware Portal 3.1 Cross Site Scripting," <https://packetstormsecurity.com/files/89559>, 2024, (Accessed on 05/11/2024).
- [57] CXSecurity, "Cryptsetup 2:1.7.3-2 Root Initramps Shell," <https://cxsecurity.com/issue/WLB-2016110132>, 2024, (Accessed on 05/11/2024).
- [58] OpenAI, "GPT-5," <https://openai.com/index/introducing-gpt-5/>, 2025, [Online; accessed 2026-07-19].
- [59] Anthropic, "Claude Opus 4.8," <https://www.anthropic.com/claude>, 2026, [Online; accessed 2026-07-19].
- [60] Google DeepMind, "Gemini 3 Pro," <https://deepmind.google/technologies/gemini/>, 2026, [Online; accessed 2026-07-19].
- [61] Qwen Team, "Qwen3 Technical Report," <https://arxiv.org/abs/2505.09388>, 2025, [Online; accessed 2026-07-19].
- [62] Seebug, "Synology Photo Station 6.7.3-3432 / 6.3-2967 - Remote Code Execution," <https://www.seebug.org/vuldb/ssvid-96600>, 2024, (Accessed on 06/02/2024).
- [63] Packet Storm, "Synology Photo Station 6.7.3-3432 / 6.3-2967 - Remote Code Execution," <https://packetstormsecurity.com/files/143745>, 2024, (Accessed on 06/02/2024).
- [64] Exploit Database, "Synology Photo Station 6.7.3-3432 / 6.3-2967 - Remote Code Execution," <https://www.exploit-db.com/exploits/42434>, 2024, (Accessed on 06/02/2024).
- [65] Vulnerability-lab, "OpenZ v3.6.60 ERP - Employee Persistent XSS Vulnerability," https://www.vulnerability-lab.com/get_content.php?id=2234, 2024, (Accessed on 06/02/2024).
- [66] Exploit Database, "OpenZ ERP 3.6.60 - Persistent Cross-Site Scripting," <https://www.exploit-db.com/exploits/48450>, 2024, (Accessed on 06/02/2024).
- [67] CXSecurity, "HomeAutomation 3.3.2 Open Redirect," <https://cxsecurity.com/issue/WLB-2019120132>, 2024, (Accessed on 06/02/2024).
- [68] NVD, "CVE-2021-4019," <https://nvd.nist.gov/vuln/detail/CVE-2021-4019>, 2026, (Accessed on 27/07/2026).
- [69] "AVideo Security Advisory GHSA-m7r8-6q9j-m2hc: incomplete fix," <https://github.com/WWBN/AVideo/security/advisories/GHSA-m7r8-6q9j-m2hc>, 2025, (Accessed on 07/26/2026).
- [70] "AVideo Security Advisory GHSA-793q-xgj6-7frp: incomplete fix," <https://github.com/WWBN/AVideo/security/advisories/GHSA-793q-xgj6-7frp>, 2025, (Accessed on 07/26/2026).
- [71] CVE, "CVE-2018-6867," <https://www.cve.org/CVERecord?id=CVE-2018-6867>, 2025, (Accessed on 20/03/2025).
- [72] —, "CVE-2019-1000018," <https://www.cve.org/CVERecord?id=CVE-2019-1000018>, 2025, (Accessed on 20/03/2025).
- [73] —, "CVE-2019-13029," <https://www.cve.org/CVERecord?id=CVE-2019-13029>, 2025, (Accessed on 20/03/2025).
- [74] —, "CVE-2021-45422," <https://www.cve.org/CVERecord?id=CVE-2021-45422>, 2025, (Accessed on 20/03/2025).
- [75] —, "CVE-2023-33140," <https://www.cve.org/CVERecord?id=CVE-2023-33140>, 2025, (Accessed on 20/03/2025).
- [76] "Sugarcrm unauthenticated remote code execution (rce) - cxsecurity.com," [Online; accessed 2025-10-25]. [Online]. Available: <https://cxsecurity.com/issue/WLB-2025100006>
- [77] "Wordpress backup migration 1.3.7: Remote command execution - cxsecurity.com," [Online; accessed 2025-10-28]. [Online]. Available: <https://cxsecurity.com/issue/WLB-2025100014>
- [78] "Packet storm," [Online; accessed 2025-10-29]. [Online]. Available: <https://packetstorm.news/help/view/8/>
- [79] "Exploit database submission guidelines," [Online; accessed 2025-10-29]. [Online]. Available: <https://www.exploit-db.com/submit/>
- [80] FIRST, "How EPSS Works: The EPSS Model," <https://www.first.org/epss/how-it-works>, 2025, (Accessed on 09/27/2026).
- [81] D. Cassel, N. Sabino, M.-C. Hsu, R. Martins, and L. Jia, "Nodemedicate: Automatic detection and exploit synthesis for node.js vulnerabilities," in *Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS'25)*. doi, vol. 10, 2025.
- [82] Q. Zhang, Y. Shen, J. Liu, Y. Xu, H. Shi, Y. Jiang, and W. Chang, "Sunflower: Enhancing linux kernel fuzzing via exploit-driven seed generation," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2025, pp. 586–596.
- [83] M.-D. Nguyen, S. Bardin, R. Bonichon, R. Groz, and M. Lemerre, "Binary-level directed fuzzing for {use-after-free} vulnerabilities," in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, 2020, pp. 47–62.
- [84] X. Zou, Y. Hao, Z. Zhang, J. Pu, W. Chen, and Z. Qian, "Syzbridge: Bridging the gap in exploitability assessment of linux kernel bugs in the linux ecosystem," in *NDSS*. Internet Society, 2024.
- [85] B. Ruan, J. Liu, C. Zhang, and Z. Liang, "Kernjc: Automated vulnerable environment generation for linux kernel vulnerabilities," in *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*, 2024, pp. 384–402.
- [86] J. Yin, M. Tang, J. Cao, and H. Wang, "Apply transfer learning to cybersecurity: Predicting exploitability of vulnerabilities by description," *Knowledge-Based Systems*, vol. 210, p. 106529, 2020.
- [87] J. Yin, M. Tang, J. Cao, M. You, H. Wang, and M. Alazab, "Knowledge-driven cybersecurity intelligence: Software vulnerability coexploitation behavior discovery," *IEEE transactions on industrial informatics*, vol. 19, no. 4, pp. 5593–5601, 2022.
- [88] Y. Dong, W. Guo, Y. Chen, X. Xing, Y. Zhang, and G. Wang, "Towards the detection of inconsistencies in public security vulnerability reports," in *28th USENIX security symposium (USENIX Security 19)*, 2019, pp. 869–885.
- [89] H. Guo, S. Chen, Z. Xing, X. Li, Y. Bai, and J. Sun, "Detecting and augmenting missing key aspects in vulnerability descriptions," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 3, pp. 1–27, 2022.
- [90] W. You, P. Zong, K. Chen, X. Wang, X. Liao, P. Bian, and B. Liang, "Semfuzz: Semantics-based automatic generation of proof-of-concept exploits," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 2139–2154.
- [91] M. Tang, J. Yin, M. Alazab, J. Cao, and Y. Luo, "Modeling of extreme vulnerability disclosure in smart city industrial environments," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 6, pp. 4150–4158, 2020.
- [92] B. Ruan, J. Liu, W. Zhao, and Z. Liang, "Vulzoo: A comprehensive vulnerability intelligence dataset," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 2334–2337.
- [93] X. Li, S. Moreschini, Z. Zhang, F. Palomba, and D. Taibi, "The anatomy of a vulnerability database: A systematic mapping study," *Journal of Systems and Software*, vol. 201, p. 111679, 2023.
- [94] R. Croft, M. A. Babar, and L. Li, "An investigation into inconsistency of software vulnerability severity across data sources," in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2022, pp. 338–348.
- [95] H. Sun, G. Ou, Z. Zheng, L. Liao, H. Wang, and Y. Zhang, "Inconsistent measurement and incorrect detection of software names in security vulnerability reports," *Computers & Security*, vol. 135, p. 103477, 2023.
- [96] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 121–133.
- [97] C. Ragkhitwetsagul and J. Krinke, "Using compilation/decompilation to enhance clone detection," in *2017 IEEE 11th international workshop on software clones (IWSC)*. IEEE, 2017, pp. 1–7.
- [98] S. Kim, S. Woo, H. Lee, and H. Oh, "Vuddy: A scalable approach for vulnerable code clone discovery," in *2017 IEEE symposium on security and privacy (SP)*. IEEE, 2017, pp. 595–614.
- [99] M. Kusner, Y. Sun, N. Kolkin, and K. Weinberger, "From word embeddings to document distances," in *International conference on machine learning*. PMLR, 2015, pp. 957–966.